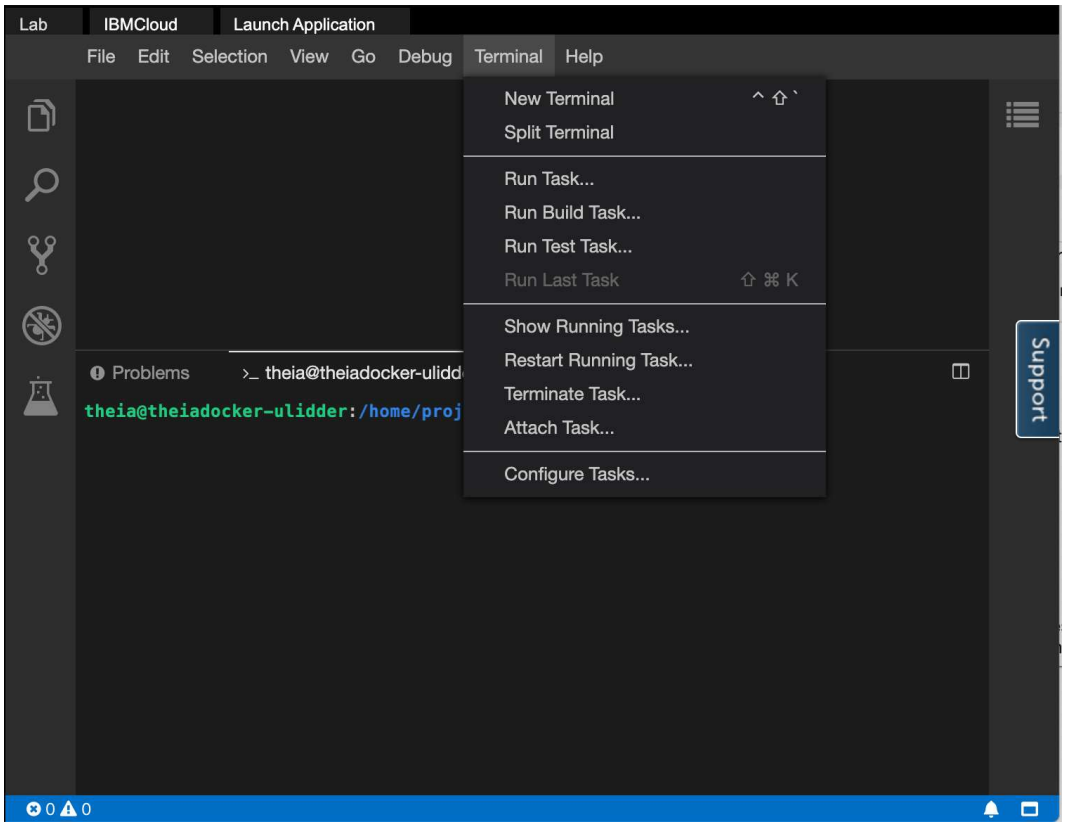
# Exercise 1: Simple HTML with React

1. Open a terminal window by using the menu in the editor: Terminal > New Terminal.




2. Change to your project folder.

```
1
cd /home/project
```

Copied! Executed!

3. Run the following command to create a HTML file named `firstReactPage.html`.

```
1
touch firstReactPage.html
```

Copied! Executed!

4. Now click the button below to open the file in edit mode.

Open **firstReactPage.html** in IDE

5. Copy and paste the following code in the editor and save the HTML file.

```
1
2
3
4
5
6
7
8
9
10
11
12
```

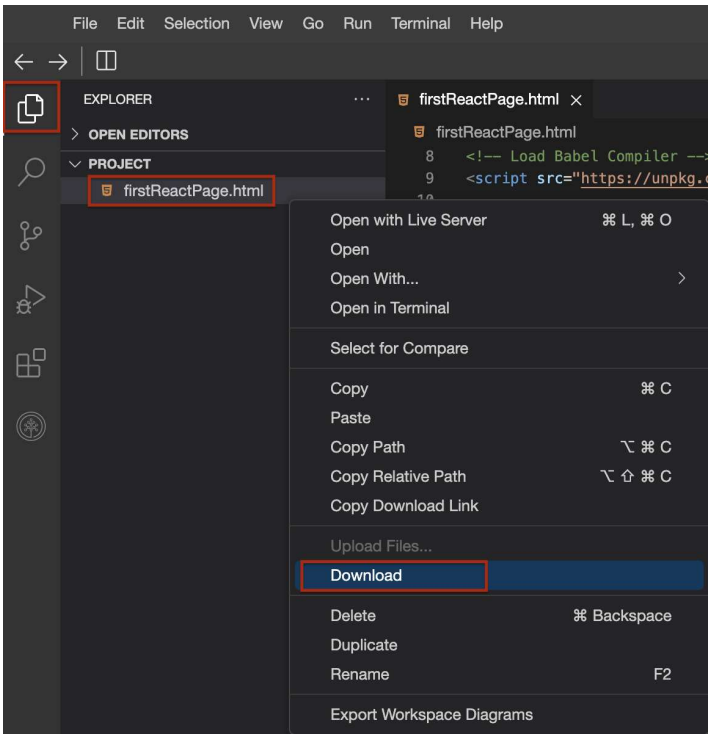
```
13.
14.
15.
16.
17.
18.
19.
20.
21.
22.
23.
24.
25.
26.
27.
```

```
1. <html>
2.
3. <!-- Load React API -->
4. <script src= "https://unpkg.com/react@18/umd/react.production.min.js"></script>
5. <!-- Load React DOM-->
6. <script src= "https://unpkg.com/react-dom@18/umd/react-dom.production.min.js"></script>
7.
8. <!-- Load Babel Compiler -->
9. <script src="https://unpkg.com/babel-standalone@6.15.0/babel.min.js"></script>
10.
11. <body>
12.
13. <!--
14. A container to display react DOM elements
15. -->
16. Hello World!
17. <div id="compl"></div>
18.
19. <script type="text/babel">
20.     const displayTime = () => {
21.         const currDate = new Date();
22.         ReactDOM.render(<p>{currDate.toString()}</p>, document.getElementById('compl'));
23.     }
24.     setInterval(displayTime, 1000);</script>
25.
26. </body>
27. </html>
```

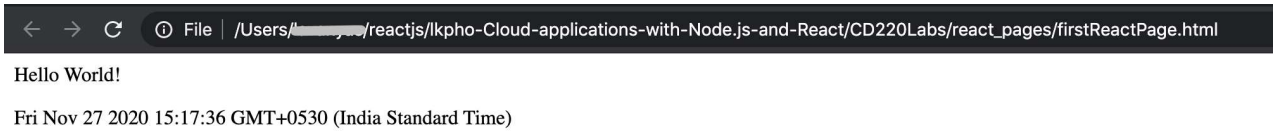
Copied!

This is only suitable for development environment.

6. Right-click on `firstReactPage.html` and click on `Download`. This will download a copy of this file in your local system.




7. Now open the downloaded file in your local browser. The page will appear as below.




# Exercise 2: First React Application

1. In the command line, run the following command to create a new React app.

```
1
npx create-react-app todoapp
```

Copied! Executed!

```
theia@theia-lavanyas:/home/project$ npx create-react-app todoapp
Need to install the following packages:
  create-react-app@5.0.1
Ok to proceed? (y)
```

2. At the prompt when it asks for a confirmation to proceed, press `y` and `enter`or `return` on the keyboard to confirm.

In a few seconds, the a directory named `todoapp` is created with all the necessary files and packages.

3. Change to the `todoapp` directory by running the following command.

```
1
cd todoapp
```

Copied! Executed!

4. Run the following command to start the React application.

```
1
npm start
```

Copied! Executed!

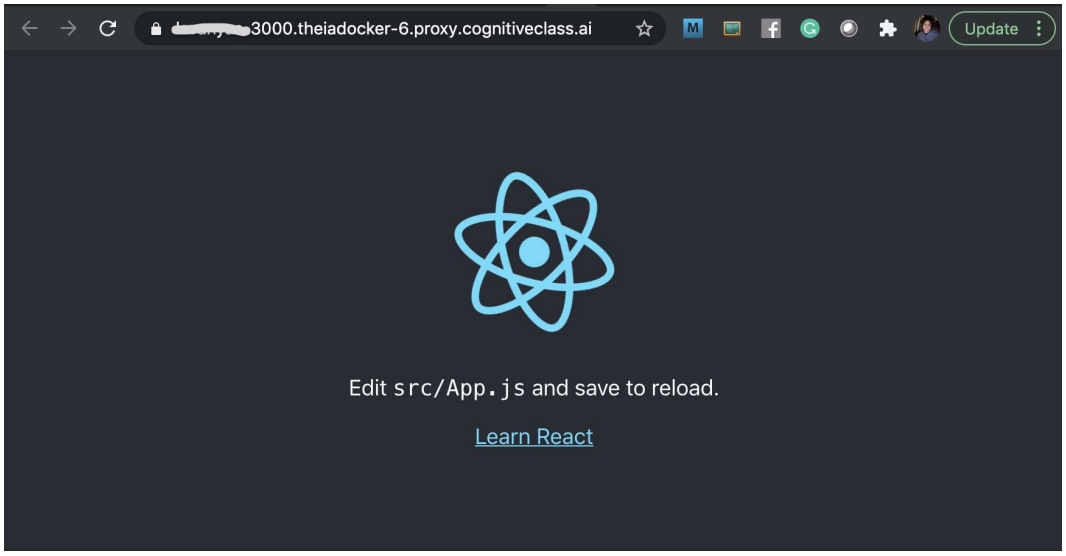
```
Compiled successfully!

You can now view todoapp in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://172.22.143.102:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

5. Click on the button below to launch the react application on the browser.

REACT Application

If you are running it in your local system open broswer and type `http://localhost:3000` in the address bar.

The todoapp UI will appear on the browser as seen in the image below.




6. Open the file App.js in the todoapp by clicking on the button below or navigating to it from the explorer `/home/project/todoapp/src/App.js` and view the content.

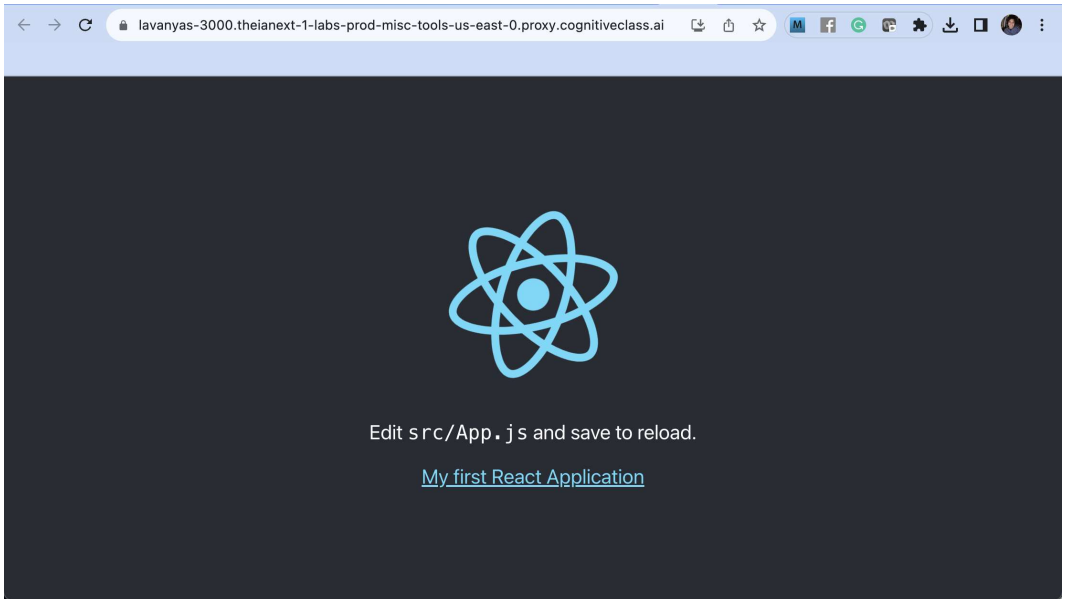
Open **App.js** in IDE

7. Change the line `Learn React` in App.js as shown below to `My first React Application` as shown below.

```
todoapp > src > App.js > ...
1   import logo from './logo.svg';
2   import './App.css';
3
4   function App() {
5     return (
6       <div className="App">
7         <header className="App-header">
8           <img src={logo} className="App-logo" alt="logo" />
9           <p>
10            Edit <code>src/App.js</code> and save to reload.
11          </p>
12          <a
13            className="App-link"
14            href="https://reactjs.org"
15            target="_blank"
16            rel="noopener noreferrer"
17          >
18            My first React Application
19          </a>
20        </header>
21      </div>
22    );
23  }
24
25  export default App;
```

When the server is running, any change made to the files is automatically picked up by the server.

8. Refresh your page to see the update.




9. To stop the server, go to the main command window and press `ctrl + c` to stop the server.

# Exercise 3: Function Components, Class Components, Properties and States

1. In the command line change to the `/home/project` directory.

```
1
cd /home/project
```

Copied! Executed!

2. Run the following command to create a new React app named `react-apps`.

```
1
npx create-react-app react-apps
```

Copied! Executed!

3. At the prompt if it asks for a confirmation to proceed, press `y` and `enter` or `return` on the keyboard to confirm.

In a few seconds, the a directory named `react-apps` is created with all the necessary files and packages.

4. Change to the `react-apps` directory by running the following command.

```
1
cd react-apps
```

Copied! Executed!

5. Click on the button below to open App.js on the editor.

Open **App.js** in IDE

6. Replace the content of App.js with the following.

```
1
```

```
2
3
4
5
6
7
8
9
```

```
function App() {
  return (
    <div >
      Hello World!
    </div>
  );
}

export default App;
```

Copied!

7. In the terminal window run the following commands one after the other.

```
1
```

```
npm start
```

Copied! Executed!

You will see this output indicating that the server is running.

8. Click on the button below to launch the react application on a browser.

REACT Application

If you are running it in your local system open broswer and type `http://localhost:3000` in the address bar.

The react-apps UI will appear on the browser. All the styling has been removed and you should see **Hello World** written on the page.

9. Now let’s start adding a component to the page. Edit `App.js` again and replace the content with the code given below.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```

```
function App(props) {
  const colorStyle = {
    color:props.color,
    fontSize:props.size
  }
  return (
    <div>
      <div style={colorStyle}>
      Hello World!
      </div>
    </div>
  );
}

```

```
export default App;
```

Copied!

We have made changes to App.js to make it a component which will take some style setting as props. We now have to set the properties.

When the server is running, any change made to the files is automatically picked up by the server.

7. To set the props of the App component, we will make changes in `index.js`. Click on the button below to open `index.js` in the editor.

Open **index.js** in IDE

8. Replace the content of `index.js` with the code given below. If you notice the root element is created in a different manner. This is how it is done from React 18 onwards.

```
1
2
3
4
5
6
7
8
```

```
import React from 'react';
import App from './App';
import { createRoot } from 'react-dom/client';

// This is how it it is done from React 18 onwards
const container = document.getElementById('root');
const root = createRoot(container);
root.render(<App color="green" size="20px"/>);
```

Copied!

As can be seen in the code we are passing the color and size.

8. Now when you refresh the broswer rendering, it will be rerendered with the style.

9. Change the color and the size in `index.js` as per your preference, save and refresh the React application on the browser page to see the changes.

# Practice Exercise

Let's set the props from outside and maintain the state of the component inside. The state we will maintain is the number of times the button is clicked.

1. Here you are going to add a state named `counter`, to keep track of the number of times the button is clicked. Replace the code in App.js with the following code and save it. Refresh the page that is rendered.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
```

```
15
16
17
18
19
20
21
```

```
import React, {useState} from 'react';

const App = (props)=> {
  const [counter, setCounter] = useState(0);

  let incrementCounter = ()=> {
    setCounter(counter+1);
  }
  let colorStyle = {color:props['color'],fontSize:props['size']}
    return (
      <div style={colorStyle}>
        React Component
        <br /><br />
        <button onClick={incrementCounter}>Click Me!</button>
        <br /><br />
        {counter}
      </div>
    );
  }

  export default App;
```

Copied!

You will see that the `counter` state is being refreshed everytime the button is clicked.

12. To stop the server, go to the main command window and press `ctrl + c` to stop the server.

# Exercise 4: Using states in functional components

In this section we will see how to do the same exercise, the state being retrieved from an asynchronous method call.

1. This script will require the axios module to run. Run the following command to install it.

```
1
```

```
npm install --save axios
```

Copied! Executed!

2. Replace the code in `App.js` with the following code. Here the fucntional component has been created using ES6 style.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
```

```
22.
23.
24.
25.
26.
27.
28.
29.
30.
31.
32.
33.
34.
35.
36.
37.
38.
39.
40.
41.
42.
43.
44.
45.
```

```
import axios from 'axios';
import React, {useState, useEffect} from "react";

const App = (props)=> {
    //define the state and the setter
    const [APIlist, setAPIlist] = useState();

    //On load invoke method to genrate list
    useEffect(() => {  let url = "https://api.publicapis.org/entries?category=Animals";
    axios({
      method: 'get',
      url: url,
      responseType: 'json'
    }).then(resp => {
        let listOfEntries = resp.data.entries;
        let listOfEntriesAsArray = Object.entries(listOfEntries);
        let i=1;
        let entryDetails = listOfEntriesAsArray.map((entryDetail)=>{
          return(
          <li key={i++}>
            <a href={entryDetail[1]["Link"]} target="_blank" rel="noreferrer">{entryDetail[1]["API"]}</a>
            </li>
          )
        })
        setAPIlist(entryDetails);
      })
    .catch(err => {
        console.log(err.toString())
    });
    }, []);


    const colorStyle = { color:props['color'], fontSize:props['size']}

  return (
      <div>
        <h2>APIs List</h2>
        <br/>

            <div style={colorStyle}><ul>{APIlist}</ul></div>
    </div>
);
}

export default App;
```

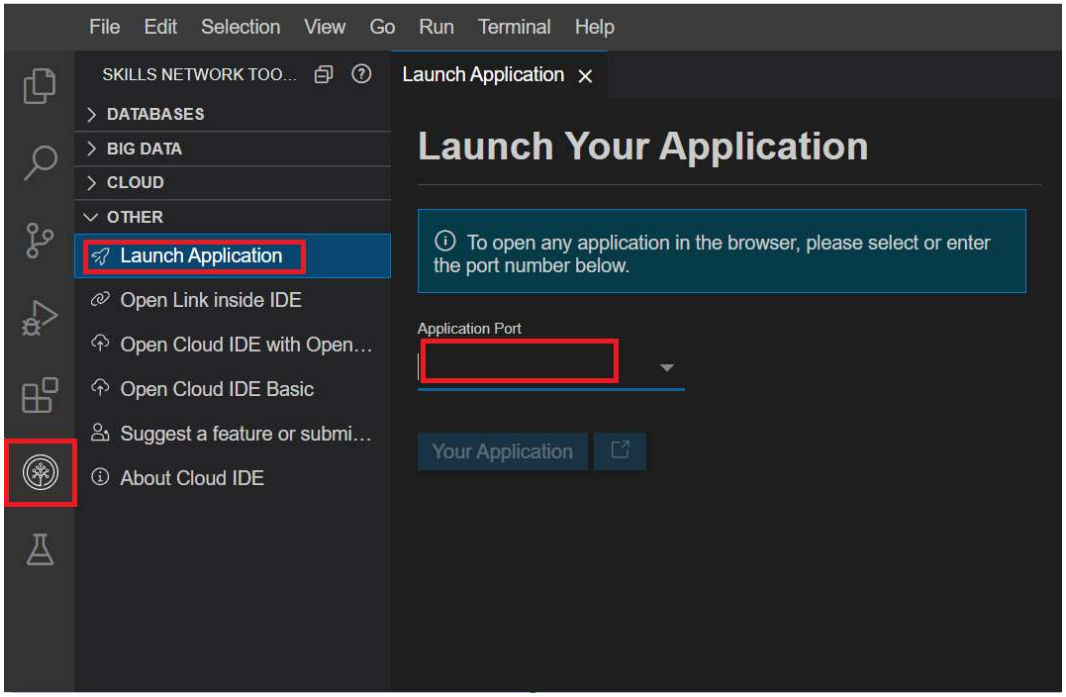
Copied!

5. Run `npm start` in the terminal to start the react app.

6. Click on the button below to open the React application on new browswer tab or click **Skills Network** button on the left, choose the **Other** option and then click **Launch Application** and

connect to port 3000.

REACT Application




The output would appear as in the image below.

---

### APIs List

- AdoptAPet
- Axolotl
- Cat Facts
- Cataas
- Cats
- Dog Facts
- Dog Facts
- Dogs
- eBird
- FishWatch
- HTTP Cat
- HTTP Dog
- IUCN
- MeowFacts
- Movebank
- Petfinder
- PlaceBear
- PlaceDog

### Task

Modify the code in App.js to use the same URL and to render the API details in the table format given below.

## APIs List

| | |
|---|---|
| AdoptAPet | https://www.adoptapet.com/public/apis/pet_list.html |
| Axolotl | https://theaxolotlapi.netlify.app/ |
| Cat Facts | https://alexwohlbruck.github.io/cat-facts/ |
| Cataas | https://cataas.com/ |
| Cats | https://docs.thecatapi.com/ |
| Dog Facts | https://dukengn.github.io/Dog-facts-API/ |
| Dog Facts | https://kinduff.github.io/dog-api/ |
| Dogs | https://dog.ceo/dog-api/ |
| eBird | https://documenter.getpostman.com/view/664302/S1ENwy59 |
| FishWatch | https://www.fishwatch.gov/developers |
| HTTP Cat | https://http.cat/ |
| HTTP Dog | https://http.dog/ |
| IUCN | http://apiv3.iucnredlist.org/api/v3/docs |
| MeowFacts | https://github.com/wh-iterabb-it/meowfacts |
| Movebank | https://github.com/movebank/movebank-api-doc |
| Petfinder | https://www.petfinder.com/developers/ |
| PlaceBear | https://placebear.com/ |
| PlaceDog | https://place.dog |
| PlaceKitten | https://placekitten.com/ |
| RandomDog | https://random.dog/woof.json |
| RandomDuck | https://random-d.uk/api |
| RandomFox | https://randomfox.ca/floof/ |
| RescueGroups | https://userguide.rescuegroups.org/display/APIDG/API+Developers+Guide+Home |
| Shibe.Online | http://shibe.online/ |
| The Dog | https://thedogapi.com/ |
| xeno-canto | https://xeno-canto.org/explore/api |
| Zoo Animals | https://zoo-animal-api.herokuapp.com/ |

▶ Click here for the solution

**Congratulations! You have completed the lab for the first week of this course.**

## Author(s)

**Lavanya**

## Changelog

| Date | Version | Changed by | Change Description |
|---|---|---|---|
| 2023-09-15 | 1.2 | Lavanya | Changed the lab exercise to make them work on local machines and theia environment |
| 2021-01-18 | 1.1 | Lavanya | Changed the lab exercise as the remote link had some issues |
| 2020-11-27 | 1.0 | Lavanya | Initial version created based videos |